

## Question 1:

---

Describe the key differences between parallel and distributed systems.

### Solution:

Parallel and distributed systems both try to speed things up, but they do it in different ways.

- **Parallel Systems:**
  - Multiple processors work **within the same machine**.
  - They share memory and are tightly coupled.
  - Communication is fast because everything is inside one system.
  - Mainly used for high-performance tasks like scientific computing.
- **Distributed Systems:**
  - Multiple computers (nodes) connected over a **network**.
  - Each node has its **own memory and CPU**.
  - Loosely coupled and communicate via messages over a network.
  - Used for scalability, fault tolerance, cloud systems, big-data processing.

---

»

## Question 2:

---

How do virtual machines help in resource management?

### Solution:

Virtual machines (VMs) make your hardware act like it has multiple independent computers on it. They help in resource management by:

- Allowing **isolation** of workloads so processes don't interfere with each other.
- Enabling **efficient utilization** of CPU, RAM, and storage by sharing them dynamically.
- Supporting **scalability**, because new VMs can be created on demand.
- Making **migration easy**; you can move VMs between servers for load balancing.
- Providing **security** since each VM is sandboxed.

»

---

### Question 3:

---

Explain the importance of hypervisors in virtualization.

#### Solution:

Hypervisors are the boss-level software that makes virtualization possible. Their importance comes from:

- They manage and allocate hardware resources to virtual machines.
- They allow multiple operating systems to run independently on one physical machine.
- They provide isolation for security and stability.
- They control execution, scheduling, memory access, and device I/O for all VMs.
- They enable features like **snapshotting**, **live migration**, and **dynamic scaling**.

---

### Question 4:

---

What are virtual machines? Differentiate between Type 1 and Type 2 hypervisors.

#### Solution:

A virtual machine (VM) is a software-based computer that emulates real hardware and runs its own OS.

#### Type 1 Hypervisor (Bare-metal):

- Runs directly on hardware.
- Faster, more stable, used in data centers.
- Examples: VMware ESXi, Hyper-V, KVM.

#### Type 2 Hypervisor (Hosted):

- Runs on top of an existing OS.
- Easier for personal use, but slower due to extra OS layer.
- Examples: VirtualBox, VMware Workstation.

## Question 5:

---

How can GPUs improve the performance of AI and machine learning tasks? Develop a GPU-based framework for real-time image processing.

### Solution:

#### How GPUs help:

- GPUs have thousands of small cores that excel at **parallel processing**.
- Matrix operations in ML and deep learning map perfectly onto GPU architecture.
- They provide massive speedups for training neural networks and running inference.

#### GPU-based framework for real-time image processing (Conceptual Design):

##### 1. Input Capture Layer

- Access camera feed in real time.
- Convert frames into GPU-friendly tensors.

##### 2. GPU Processing Layer (CUDA / OpenCL / TensorRT)

- Load preprocessing kernels (resize, crop, normalize).
- Apply deep learning models (CNNs, YOLO, or custom CNN).
- Use batch or stream-based CUDA pipelines for low latency.

##### 3. Memory Optimization

- Keep all tensors in GPU memory.
- Use pinned memory to avoid slow transfers.

##### 4. Model Execution Engine

- Use TensorRT for optimized inference.
- Use FP16 or INT8 precision for faster performance.

##### 5. Output Rendering Layer

- Draw bounding boxes, labels, or segmentation maps.
- Display processed frames at 30–60 FPS.

##### 6. Framework Flow:

Camera → Frame Buffer → GPU Preprocessing → GPU Inference → GPU Post-processing → Output Window.

## Question 6:

---

Why does C-LOOK perform better than LOOK in some scenarios?

### Solution:

C-LOOK works like a disciplined version of LOOK. It scans in one direction, serves all requests, then **jumps directly** to the start of the next request without servicing anything on the return trip. Because of this:

- It offers **more uniform wait times**.
- It avoids extra head movement caused by scanning back through already serviced areas.
- It reduces the **overall seek time** when requests are clustered.

In busy systems with scattered I/O requests, that predictable one-way sweep often makes C-LOOK faster than LOOK.



## Question 7:

---

Compare the architectures of CPU and GPU in terms of processing efficiency.

### Solution:

#### CPU Architecture:

- Few powerful cores (4 to 16 typically).
- Designed for sequential tasks and complex logic.
- Large caches and strong single-thread performance.
- Great for OS operations, branching, and tasks needing high control flow.

#### GPU Architecture:

- Thousands of smaller, simpler cores.
- Designed for parallel workloads.
- High throughput but weaker single-core performance.
- Ideal for matrix operations, rendering, ML training, scientific computing.

#### Efficiency Difference:

CPUs shine in **low latency**, GPUs shine in **massive parallel throughput**.

---

## Question 8:

---

How can GPUs improve the performance of AI and machine learning tasks?

### Solution:

GPUs accelerate ML because:

- Neural networks rely heavily on **matrix multiplications**, which GPUs parallelize across thousands of cores.
- Training models becomes **up to 50–100 times faster** compared to CPUs.
- GPUs handle **vectorized operations** and **batch processing** efficiently.
- Frameworks like CUDA, cuDNN, TensorRT squeeze every drop of performance.

This turns AI workloads from all-nighters into “let me grab a coffee while it trains.”

---

## Question 9:

---

A company wants to implement virtualization for cost efficiency. Suggest a suitable hypervisor and justify your choice.

### Solution:

For enterprise-level cost efficiency, **Type 1 hypervisors** are the best choice.

A strong recommendation is:

**Hypervisor: VMware ESXi or KVM**

### Why:

- Runs directly on hardware for better performance.
- Highly secure and stable for multi-VM environments.
- Supports resource pooling and load balancing.
- Easy VM migration and scaling.
- KVM is open source, which helps cut costs even more.

If they want **no licensing cost**, go with **KVM**.

If they want a polished system with professional support, **ESXi** is solid.



## Question 10:

---

Which is better for handling large-scale computations: a parallel system or a distributed system? Justify your answer.

### Solution:

It depends on what “large-scale” means:

- If you want **massive raw performance** for scientific simulations, numeric computing, or deep learning, a **parallel system** (shared memory, multi-core setup) is faster and more efficient.
- If you want **scalability**, fault tolerance, and the ability to spread work across thousands of machines, then **distributed systems** win.

### General rule:

For truly large-scale computations (big data, cloud-scale tasks), **distributed systems** are usually better because you can keep adding nodes endlessly. Parallel systems are fast but limited by the size of one machine.



## Question 11:

---

Design a basic distributed system model for a real-world application (online banking).

### Solution:

**Goal:** reliable, secure, low-latency banking: authenticate users, show balances, perform transfers, logging, auditing, and high availability.

### High-level components

1. **Client layer**
  - Mobile apps and web browsers.
  - TLS for transport security.
2. **API Gateway / Load Balancer**
  - Accepts client requests, performs SSL termination, rate limits, basic auth checks, routes to services.
3. **Authentication & Authorization Service**
  - Issues tokens (OAuth2 / JWT), enforces MFA and session policies.
4. **Frontend Service (Stateless)**
  - Handles UI logic and talks to backend microservices via the API gateway.
5. **Account Service (Stateful)**
  - Holds account metadata; exposes read/write APIs; backed by a strongly consistent database.
6. **Transaction Service (Stateful, transactional)**
  - Handles debits/credits, ensures ACID semantics for money movement.
  - Uses distributed transactions or a ledger pattern (append-only log) with idempotent operations.
7. **Ledger / Audit Store**
  - Immutable append-only store or event log for every financial operation for audit and reconciliation.
8. **Fraud Detection Service**
  - Streams transactions (near real time) to ML models; raises alerts.
9. **Notification Service**
  - Sends SMS / email / push notifications for events.
10. **Cache Layer**
  - Read caches (Redis) for balances and non-sensitive hot data.
11. **Data Storage**
  - Primary DBs: transactional DB (strong consistency, e.g., relational), read replicas for reporting.
  - Analytics store for batch processing.
12. **Monitoring & Observability**
  - Metrics, distributed tracing, centralized logs, alerts.
13. **Replication & Failover**
  - Active-passive or active-active across availability zones for high availability.
14. **Security & Compliance**
  - End-to-end encryption, key management, role-based access controls, audit trails, PCI/other compliance.

### Interaction flow (example: transfer money)

1. Client → API gateway (HTTPS).
2. Gateway → Auth service validates token.
3. Gateway routes to Transaction Service.
4. Transaction Service: Reserve/lock source account, debit, credit destination (either via a distributed transaction or ledger + compensating events).
5. Write to Ledger / Audit store.
6. Notify Notification Service and update caches.
7. Commit / release locks; return response.

### Design notes / trade-offs

- **Consistency**: Money operations must be strongly consistent. Prefer transactions, or use a ledger + compensation if loosening latency is required.
- **Availability**: Use geo-replication + quorum settings tuned for trade-off between availability and consistency.
- **Scalability**: Stateless frontends + microservices, scale horizontally; partition account space to distribute load.
- **Failure handling**: Idempotency keys, retries, dead-letter queues for failed events.
- **Security**: Minimal trust between services, network segmentation, strict logging and alerts.



---

## Question 12:

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The FCFS scheduling algorithm is used. The head is initially at cylinder number 53. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is?

### Solution:

We service requests in the given order under FCFS. Compute absolute movement step by step.

Start head = 53.

1.  $53 \rightarrow 98 = |98 - 53| = 45$
2.  $98 \rightarrow 183 = |183 - 98| = 85$



$$3. 183 \rightarrow 41 = |41 - 183| = 142$$

$$4. 41 \rightarrow 122 = |122 - 41| = 81$$

$$5. 122 \rightarrow 14 = |14 - 122| = 108$$

$$6. 14 \rightarrow 124 = |124 - 14| = 110$$

$$7. 124 \rightarrow 65 = |65 - 124| = 59$$

$$8. 65 \rightarrow 67 = |67 - 65| = 2$$

Now add them precisely:

$$45 + 85 = 130$$

$$130 + 142 = 272$$

$$272 + 81 = 353$$

$$353 + 108 = 461$$

$$461 + 110 = 571$$

$$571 + 59 = 630$$

$$630 + 2 = 632$$

**Total head movement = 632 cylinders.**

---

»

---

## Question 13:

---

Consider a disk with 200 tracks and the queue has random requests from different processes in the order: 55, 58, 39, 18, 90, 160, 150, 38, 184. Initially arm is at 100. Find the Average Seek length using FIFO, SSTF, SCAN and C-SCAN algorithm.

**Solution:**

Number of requests = 9. Initial head = 100. I assume the head initially moves toward the lower-numbered cylinders (toward 0) for SCAN and C-SCAN calculations. If you want the other direction, tell me and I will recompute.

1. **FIFO (FCFS)** — service in given order:

Movements:

$$100 \rightarrow 55 = 45$$

$$55 \rightarrow 58 = 3$$

$$58 \rightarrow 39 = 19$$

$$39 \rightarrow 18 = 21$$

$$18 \rightarrow 90 = 72$$

$90 \rightarrow 160 = 70$

$160 \rightarrow 150 = 10$

$150 \rightarrow 38 = 112$

$38 \rightarrow 184 = 146$

Sum = 498 cylinders.

Average seek length =  $498 / 9 = 55.333$  cylinders.

2. **SSTF (Shortest Seek Time First)** — at each step pick closest pending request. One possible servicing order and total movement calculation yields total movement = 248 cylinders.

Average seek length =  $248 / 9 = 27.556$  cylinders.

3. **SCAN** (assume initial direction toward 0) — head services requests toward decreasing cylinder numbers, goes to cylinder 0, then reverses and services remaining requests. Total movement = 284 cylinders.

Average seek length =  $284 / 9 = 31.556$  cylinders.

4. **C-SCAN** (assume initial direction toward 0) — head moves toward 0 servicing requests, when it reaches 0 it jumps to the end (199) and continues servicing in the same logical direction. Counting the physical head movement including the jump gives total movement = 348 cylinders.

Average seek length =  $348 / 9 = 38.667$  cylinders.

#### Summary (assuming initial direction down toward 0):

- FIFO average = **55.333**
- SSTF average = **27.556**
- SCAN average = **31.556**
- C-SCAN average = **38.667**

If you want the SCAN/C-SCAN results for the head initially moving up instead, say so and I will recalc.



---

## Question 14:

Disk requests come to a disk driver for cylinders in the order 10, 22, 20, 2, 40, 6 and 38 at a time when the disk drive is reading from cylinder 20. The seek time is 6 ms/cylinder. The total seek time, if the disk arm scheduling algorithm is first-come-first-served is...

### Solution:

Start head = 20. Sequence to service: 10, 22, 20, 2, 40, 6, 38.

Compute movements step by step:

1.  $20 \rightarrow 10 = |10 - 20| = 10$
2.  $10 \rightarrow 22 = |22 - 10| = 12$
3.  $22 \rightarrow 20 = |20 - 22| = 2$
4.  $20 \rightarrow 2 = |2 - 20| = 18$
5.  $2 \rightarrow 40 = |40 - 2| = 38$
6.  $40 \rightarrow 6 = |6 - 40| = 34$
7.  $6 \rightarrow 38 = |38 - 6| = 32$

Total cylinders moved =  $10 + 12 + 2 + 18 + 38 + 34 + 32 = 146$  cylinders.

Seek time per cylinder = 6 ms.

Total seek time =  $146 \times 6 = 876$  ms.

---

~

### Question 15:

---

Consider the disk system with 100 cylinders. The request to access the cylinders occur in the following sequence: 4, 37, 10, 7, 19, 73, 2, 15, 6, 20. Assuming the head is currently at cylinder 50 what is the time taken to satisfy all requests if it takes 1 ms to move from one cylinder to adjacent one and shortest seek time first algorithm (SSTF) is used.

### Solution:

Requests = [4, 37, 10, 7, 19, 73, 2, 15, 6, 20]

Initial head = 50. Time per cylinder = 1 ms.

Apply SSTF (choose nearest pending each step). The calculated total head movement = **119 cylinders**.

Time = movement  $\times$  1 ms = **119 ms**.

Detailed note: SSTF selected the closest cylinder at every step, producing a total of 119 cylinder movements and therefore 119 milliseconds total.

~

Given:

Requests: 98, 183, 37, 122, 14, 124, 65, 67

Initial head: 53

Tracks: 0 to 199

Head direction for SCAN/C-SCAN: Right

## 16. FCFS (First-Come, First-Served)

Service in the exact order given.

Start at 53:

1.  $53 \rightarrow 98 = 45$
2.  $98 \rightarrow 183 = 85$
3.  $183 \rightarrow 37 = 146$
4.  $37 \rightarrow 122 = 85$
5.  $122 \rightarrow 14 = 108$
6.  $14 \rightarrow 124 = 110$
7.  $124 \rightarrow 65 = 59$
8.  $65 \rightarrow 67 = 2$

Add them carefully:

$45 + 85 = 130$   
 $+146 = 276$   
 $+85 = 361$   
 $+108 = 469$   
 $+110 = 579$   
 $+59 = 638$   
 $+2 = 640$

**Total FCFS movement = 640 tracks**

## 17. SSTF (Shortest Seek Time First)

---

Pick the nearest request to the current head each time.

Start head = 53

Pending = {98,183,37,122,14,124,65,67}

Step-by-step choices:

1. Nearest to 53 is **65** (distance 12)  
 $53 \rightarrow 65 = 12$
2. Nearest to 65 is **67** (2)  
 $65 \rightarrow 67 = 2$
3. Nearest to 67 is **65 already done**, next closest **98** (31)  
 $67 \rightarrow 98 = 31$
4. Nearest to 98 is **122** (24)  
 $98 \rightarrow 122 = 24$
5. Nearest to 122 is **124** (2)  
 $122 \rightarrow 124 = 2$
6. Nearest to 124 is **183** (59)  
 $124 \rightarrow 183 = 59$
7. Nearest to 183 is **37** (146), **14** (169)  $\rightarrow$  choose **37**  
 $183 \rightarrow 37 = 146$
8. Last pending is **14**  
 $37 \rightarrow 14 = 23$

Now total:

$12 + 2 + 31 + 24 + 2 + 59 + 146 + 23 = \mathbf{299}$

**Total SSTF movement = 299 tracks**



## 18. SCAN (Elevator) — direction RIGHT

---

Head moves right first, servicing requests  $\geq 53$ , then turns around.

Requests on right side: **65, 67, 98, 122, 124, 183**

Requests on left side: **37, 14**

SCAN order (right direction):

53 → 65 → 67 → 98 → 122 → 124 → 183 → then to track 199 → then reverse → 37 → 14

Movements:

1. 53 → 65 = 12
2. 65 → 67 = 2
3. 67 → 98 = 31
4. 98 → 122 = 24
5. 122 → 124 = 2
6. 124 → 183 = 59
7. 183 → 199 = 16 (go to end because SCAN hits boundary)
8. Reverse: 199 → 37 = 162
9. 37 → 14 = 23

Now add:

12 + 2 + 31 + 24 + 2 + 59 + 16 + 162 + 23 = 331

Total SCAN movement = 331 tracks

---

## 19. C-SCAN (Circular SCAN) — direction RIGHT

---

Head moves right only, reaches the end, jumps to 0, continues servicing.

Right side requests: 65, 67, 98, 122, 124, 183

Left side requests: 37, 14

Order:

53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → jump to 0 → 14 → 37

Movements:

1. 53 → 65 = 12
2. 65 → 67 = 2
3. 67 → 98 = 31
4. 98 → 122 = 24
5. 122 → 124 = 2

6.  $124 \rightarrow 183 = 59$
7.  $183 \rightarrow 199 = 16$
8. Jump:  $199 \rightarrow 0 = 199$
9.  $0 \rightarrow 14 = 14$
10.  $14 \rightarrow 37 = 23$

Add them:

$$12 + 2 + 31 + 24 + 2 + 59 + 16 + 199 + 14 + 23 = 382$$

**Total C-SCAN movement = 382 tracks**



## Final Results Summary

---

| Algorithm | Total Head Movement |
|-----------|---------------------|
| FCFS      | 640                 |
| SSTF      | 299                 |
| SCAN      | 331                 |
| C-SCAN    | 382                 |



## Question 20:

---

Explain the impact of disk architecture on system performance.

### Solution:

Disk architecture decides how fast data can be accessed, and that spills directly into system performance. A few factors shape this:

- **Seek time:** how quickly the read/write head can reach the correct track. Long seek times slow everything.

- **Rotational latency:** waiting for the platter to spin to the needed sector. Higher RPM gives lower delay.
- **Data density:** more bits per track means more data read per rotation.
- **Cache size on the disk:** reduces physical reads and boosts responsiveness.
- **Interface type (SATA, NVMe):** faster interfaces handle higher data throughput.
- **Scheduling support:** drives optimized for sequential reads outperform on workloads with predictable patterns.

When the architecture is efficient, the OS spends less time waiting and more time doing actual work.



## Question 21:

---

How does disk cache improve the performance of storage devices?

### Solution:

Disk cache is a small amount of fast memory inside the drive. It helps in a few ways:

- **Read-ahead:** the disk loads extra data it expects you will need soon. If you do, it's instant.
- **Write-back buffering:** writes are stored in cache first so the OS doesn't have to wait for slow physical writes.
- **Reduced head movement:** caching clusters small, random I/O into larger sequential ones.
- **Decreased latency:** frequently accessed blocks stay in the cache, avoiding repeated disk seeks.

This basically makes slow mechanical disks feel less prehistoric.



## Question 22:

---

What is disk scheduling? Explain its need in operating systems.



### Solution:

Disk scheduling is the method the OS uses to decide in what order disk I/O requests should be served.

Why it's needed:

- Physical disk access, especially on HDDs, is slow compared to CPU and RAM.
- Random requests cause massive seek times if not reordered intelligently.
- Good scheduling reduces total head movement.
- It improves **throughput**, **response time**, and **overall system performance**.
- Without it, the disk would serve requests purely in arrival order, often wasting time bouncing across tracks.

In short, scheduling keeps the system from spending half its life waiting on the disk.



## Question 23:

---

Describe the First-Come, First-Served (FCFS) disk scheduling algorithm with an example.

### Solution:

FCFS processes disk requests exactly in the order they arrive. There's no optimization at all. Simple but not always smart.

### Example:

Requests: 40, 10, 22, 90

Initial head: 50

Movements:

$50 \rightarrow 40 = 10$

$40 \rightarrow 10 = 30$

$10 \rightarrow 22 = 12$

$22 \rightarrow 90 = 68$

Total movement = **120 tracks**

FCFS is fair, but the head may zigzag wildly, making performance pretty average.

## Question 24:

---

Explain the Shortest Seek Time First (SSTF) scheduling algorithm with a diagram.

### Solution:

SSTF always chooses the pending request that is closest to the current head position. It minimizes movement on each step, reducing overall seek time.

### Conceptual diagram (text version):

Requests: 15, 30, 10, 5 Head at: 20 Track line: 0 ---5---10---15---20---30---40  
Steps: 1. From 20 → nearest is 15 (distance 5) 2. From 15 → nearest is 10 (5) 3.  
From 10 → nearest is 5 (5) 4. From 5 → last is 30 (25)

Total movement =  $5 + 5 + 5 + 25 = 40$

SSTF performs better than FCFS, but may cause starvation if far-away requests never become the nearest.